CogACT Numerical Alignment for $\pi_{0.5}$ / LIBERO

Updated: 2026-10-29. Recipe: `cogact_reference_v3`, checkpoint format 3.

This replaces the earlier **partial** `cogact` profile. Use `--profile cogact` explicitly. Historical `quantvla` and `legacy` algorithms remain available. The runner rejects old calibration recipes: start from the original weights in a new output directory. Original CogACT source files were not modified.

Implementation and provenance

`cogact_primitives.py` contains dependency-closed, **unmodified definitions** from the local `DA-PTQ/vla/quantvla_ptq.py`. `cogact_sampling.py` contains the original action-XY quota sampler from `DA-PTQ/scripts/quantize_quantvla.py`. `scripts/sync_cogact_primitives.py` regenerates both modules with source hashes. The server needs neither the sibling checkout nor Prismatic. Do not edit these generated definitions by hand. The $\pi_{0.5}$ interface is implemented in `cogact.py`.

Component	Revision-3 behavior
Quantized modules	Language and eligible action-expert MLP projections; language/expert Q/K/V/O excluded from W/A QDQ
Fixed boundaries	First expert block eligible by default; at least the final two complete expert blocks always bypass W/A QDQ
Rotations	Original SVD, zigzag permutation, source-dtype matrices/operations, blocks 16/16, output restore, smoothing 0.15
Rotation cache	Original weight/config-validated cache reused across calibration variants; saved deployment buffers do not depend on the cache
Bias convention	Original reference restore-mode bias behavior retained; no silent correction of the source algorithm
Weight masters	Original CPU `weight_master`/`bias_master`; adaptive assignment and compensation requantize from masters
General weight QDQ	Original per-output-channel RTN or MSE
Adaptive weight QDQ	Original per-output-row RTN, even when general mode is MSE
Sensitivity probe	W-QDQ student, A quantization off, per-batch structural weights, seed `calibration_seed + 200000 + step`
Allocation	Global output channels, rounded-up target, original `score >= kth_threshold` tie behavior
Fallback	W16 rows retain A8; excluded modules retain original floating precision

Component	Revision-3 behavior
Activation observer	Original percentile/QDQ implementation, 99.9th percentile, 64 calls, first 4096 rows/call
Unobserved A scale	Original dynamic absmax fallback until channel scales are observed
Structural prior	Original seven virtual links and per-batch formula
Attention statistics	Mean of per-sample population standard deviations, averaged over calls; mean of per-call output RMS
ATM	Original clipping `[0.5,2]` and neutral threshold 0.02
OHB	Original log-clipped teacher/student ratio; **inverse beta** folded into O and its bias
CSRC	Original grouped affine or weighted identity-prior dense solver, original matrix orientation and folding; no quartic/LBFGS fit
Stage order	Teacher → W-QDQ wrappers → student probe → channel assignment → student statistics with A observation/QDQ → ATM/OHB from the same statistics → CSRC → final activation-only calibration
Sampling	Original action-XY quota/warmup/fallback algorithm on a seeded LeRobot stream; task balancing and first-five-episode restriction removed by default
Seed	3407 by default
Deployment	Weights, channel assignments, SVD buffers/strides, A scales, biases and optional bypass buffers saved; inference does not recalibrate or re-QDQ

Budget interpretation

30% now means **output channels**, as in CogACT, rather than whole Linear layers. Threshold ties can retain more than the target. Rows have different parameter costs when their input widths differ: this is **not an exact equal-parameter budget**. Random controls use distinct shuffled priorities and select exactly the rounded-up target. Inspect `channel_allocation`, `high_precision_channels` and `high_parameter_fraction`; do not reuse the previous whole-layer-budget claim.

CSRC enablement and parameters

The local CogACT entrypoint defaults affine compensation to **off** and forcibly disables dense compensation and its FP16 bypass. The requested compensated controls explicitly enable compensation; this is an experiment choice.

- `--csrc-method affine` uses the original grouped moment correction. Defaults follow the README's optional affine recipe: group size 32, shrink 0.55, scales `[0.5,2]`, shift limit 3 teacher standard deviations, neutrality 0.02.
- `--csrc-method dense` explicitly enables the original dense function that the local entrypoint otherwise disables. Defaults: ridge `1e-4`, min eigenvalue `1e-6`, weight gain 1, feature weight 1, identity weight 8, solver fallback `rcond=1e-3`, low-rank structure, rank 8, delta clip 0.25.
- `--csrc-bypass-enabled` enables the source's optional bypass for PTQLinear targets. Ordinary floating targets use direct folding. Default: off.

- `--no-csrc` matches the source entrypoint's default compensation enablement.

Historical `--csrc-max-iter` is an LBFGS option and does not govern these solvers. Use separate output roots for affine, dense, and no-compensation experiments.

Necessary model and experiment interfaces

Numerical alignment does not make CogACT and π₀.₅ identical models:

1. π₀.₅ predicts flow velocity and uses shared prefix K/V; CogACT predicts diffusion noise and has a separate `z_embedder.linear`. CSRC therefore targets floating prefix K/V before RoPE/cache insertion. Biasless π₀.₅ targets receive an exactly zero bias before teacher collection, allowing the unchanged folding routine to represent shifts. Added biases are saved and restored.
2. π₀.₅ has GQA and keys shared with language queries. Expert Q is multiplied by alpha, giving the same per-head logit multiplier as Q and repeated K each multiplied by `sqrt(alpha)`, including prefix keys. Shared language queries are unaffected. Weight tensors and BF16 rounding are not bit-identical to CogACT's fused Q/K/V. Masked π₀.₅ attention entries are excluded from statistics.
3. LIBERO/LeRobot, original π₀.₅ normalization and the action horizon replace Bridge/RLDS. The action-XY quota algorithm is identical, but the data stream differs. Calibration samples are frames with action chunks, not rollouts.
4. W2A8, 30% W16 fallback and compensation enablement are explicit experiment settings. The local README example uses W4A8. Matching code does not imply that W2A8 will recover a published success rate.

Validation

CPU tests check the copied definitions against the actual source AST, FP32/BF16 rotated and mixed-bit forwards, both compensation solvers and master-weight folding, threshold ties, inverse OHB, per-call statistics and the original sampler. Tiny models exercise actual joint-Gemma flow control, all allocation modes, both CSRC choices, fixed tail protection and exact export/reload. Tests reject old calibration recipes. Source-comparison tests skip if the separate reference checkout is absent. Full-checkpoint LIBERO success and GPU performance still require server evaluation.

Server commands: calibration and evaluation separately

Update the server checkout first. Keep the existing cu128/Transformers patch; do not run `uv sync`. These commands use **GPU 3**, one calibration process and at most two evaluation models. Stop older GPU-3 launchers before starting; the worker limit cannot constrain another launcher.

Shared environment and arguments

```
cd /home/xusiyuan/openpi
export PTQ_DATA_ROOT=/data2/xusiyuan
export RUNS="$PTQ_DATA_ROOT/runs/cogact_reference_v3_w2a8_20261029"
export LIBERO_ENV=/home/xusiyuan/.conda/envs/libero-cpython
export LIBERO_PYTHON="$LIBERO_ENV/bin/python"
export LD_LIBRARY_PATH="$LIBERO_ENV/lib${LD_LIBRARY_PATH:+:$LD_LIBRARY_PATH}"
export LIBERO_CONFIG_PATH="$PWD/.libero_config"
export MUJOCO_GL=osmesa PYOPENGL_PLATFORM=osmesa LIBGL_ALWAYS_SOFTWARE=1
```

```
export LP_NUM_THREADS=8 OMP_NUM_THREADS=2
export JAX_PLATFORMS=cpu XLA_PYTHON_CLIENT_PREALLOCATE=false
export CUDA_DEVICE_ORDER=PCI_BUS_ID
export PYTHONPATH="$PWD/src:$PWD/packages/openpi-
client/src${PYTHONPATH:+:$PYTHONPATH}"
unset MUJOCO_EGL_DEVICE_ID
mkdir -p "$RUNS/logs"

COMMON=(
  --checkpoint-dir "$PTQ_DATA_ROOT/checkpoints/pi05_libero_pytorch"
  --dataset-root "$PTQ_DATA_ROOT/datasets/physical-intelligence/libero"
  --output-dir "$RUNS"
  --profile cogact --weight-bits 2 --activation-bits 8
  --weight-scale-mode mse --bf16-ratio 0.30 --fallback-activation-bits 8
  --skip-first-expert-blocks 0 --skip-last-expert-blocks 2
  --calibration-samples 512 --batch-size 1 --probe-steps 16 --seed 3407
  --max-calibration-episodes-per-task 0
  --rotation-block-size 16 --rotation-output-block-size 16
  --activation-percentile 99.9 --activation-calibration-batches 64
  --quantvla-calibration-steps 512 --matching-neutral-band 0.02
  --csrc-method dense --csrc-group-size 32 --csrc-group-shrink 0.55
)
```

To use the source dense solver, change `affine` to `dense` and choose another new root before calibration. For W4A8 diagnostics, change weight bits to 4 and also choose another root. Recreate the same `COMMON` array in a new terminal.

Calibration

```
."${COMMON[@]}" --gpus 3 --workers-per-gpu 1 --stage calibrate --dry-run

CALvenv/bin/python scripts/run_da_ptq_controls.py \
  _LOG="$RUNS/logs/calibrate_$(date +%Y%m%d_%H%M%S).log"
nohup .venv/bin/python -u scripts/run_da_ptq_controls.py \
  "${COMMON[@]}" --gpus 3 --workers-per-gpu 1 --stage calibrate \
  > "$CAL_LOG" 2>&1 < /dev/null &
echo "Calibration PID: $!"
echo "Calibration log: $CAL_LOG"
```

The runner prints `calibration_*/`; per-worker details are in its `gpu3.log`. Completed artifacts are in `$RUNS/quantized/<variant>/`. Each variant fits its own correction using the same selected frames. Compatible completed artifacts are reused; old/incompatible artifacts are rejected. The source computes full SVDs, so the first wrapping pass can be slow. Per-layer W-QDQ/SVD progress and cache hits are logged. The default cache is `.cache/cogact_reference_v3`; keep it between variants. `--no-rotation-cache-enabled` disables it, and `--rotation-cache-dir` selects another location.

Evaluation

After calibration completes, first run a Spatial diagnostic:

```
SMOKE_LOG="$RUNS/logs/spatial_smoke_$(date +%Y%m%d_%H%M%S).log"
nohup .venv/bin/python -u scripts/run_da_ptq_controls.py \
  "${COMMON[@]}" --stage evaluate --gpus 3 --workers-per-gpu 2 --split-suites
\
  --eval-python "$LIBERO_PYTHON" --base-port 2914 \
  --variants da_ptq equal_weight random_seed_0 --include-bf16 \
  --suites libero_spatial --num-trials 1 --seeds 7 \
  --save-video --video-max-trials 1 \
  > "$SMOKE_LOG" 2>&1 < /dev/null &
echo "Smoke PID: $!"
echo "Smoke log: $SMOKE_LOG"
```

Inspect outcomes and videos. If BF16 succeeds while quantized variants still consistently fail, investigate before scheduling 50 trials/task. After the diagnostic finishes and passes, run the formal evaluation:

```
FORMAL_LOG="$RUNS/logs/formal_$(date +%Y%m%d_%H%M%S).log"
nohup .venv/bin/python -u scripts/run_da_ptq_controls.py \
  "${COMMON[@]}" --stage evaluate --gpus 3 --workers-per-gpu 2 --split-suites
\
  --eval-python "$LIBERO_PYTHON" --base-port 2914 \
  --variants da_ptq equal_weight random_seed_0 random_seed_1 random_seed_2 \
  --include-bf16 --suites libero_spatial libero_object libero_goal libero_10 \
  --num-trials 50 --seeds 7 --save-video --video-max-trials 2 \
  > "$FORMAL_LOG" 2>&1 < /dev/null &
echo "Formal PID: $!"
echo "Formal log: $FORMAL_LOG"
```

Do not overlap smoke and formal launchers. Formal evaluation is 2,000 episodes/model, 12,000 across BF16 and five quantized models. Each evaluation gets a fresh directory. Success rates and paired NPZ traces are produced together; no second evaluation is needed for trajectory deviation. Read `evaluation_*/results.json`; worker logs, per-suite summaries, traces and selected videos are under that invocation.